

PROJEKTBERICHT · BENUTZERBEREICH

Terra

Visuelle Budget-App: Geld wird einer Insel mit Kategorien zugeordnet. Dieses Dokument beschreibt die zentralen Bedienelemente mit Originalfarben und Design der Anwendung – inklusive kurzer Backend-Übersicht.

Dokument: Bedienungsanleitung der UI

App: Terra (Expo / React Native)

Sprache der App-UI: Englisch (Beschreibungen hier auf Deutsch)

VERFASSER

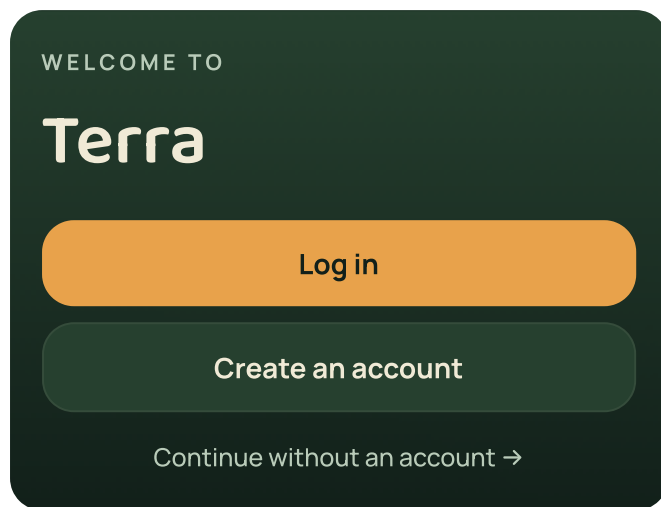
Name: Ali Alkharazi

Matrikelnummer: 5015525

Hochschule: HTW Saar

1. Startbildschirm (Anmeldung)

Beim ersten Öffnen wählt man, ob mit Konto (Backend) oder lokal offline gearbeitet wird.



Log in

Log in

Anmeldung mit E-Mail und Passwort am Backend.
Danach sind Backup und Wiederherstellung verfügbar.

Create an account

Create an account

Neues Konto anlegen. Die App speichert Daten lokal
und kann sie bei Bedarf in die Cloud sichern.

Continue without an account →

Continue without an account

Offline-Modus: Budget und Insel laufen vollständig
auf dem Gerät, ohne Server.

2. Hauptbildschirm (Welt / Insel)

Übersicht des verfügbaren Geldes, der Kategorie-Gebäude und der Aktionsleiste unten. Die Darstellung unten nutzt dieselben Farben wie in der App.



3. Elemente oben und auf der Insel

Tippen auf Betrag, Gebäude oder Flugzeug öffnet die jeweilige Funktion.

Offline

Offline / Account

Oben links. Im Offline-Modus nur Statusanzeige. Nach Login: Dialog mit **Backup** (Stand speichern), **Restore** (wiederherstellen) und **Log out**.

2026-09

1.240 €

● Insights 78% stabil

Gesamtbetrag & Insights

Tippen auf den großen Betrag öffnet die Auswertungen (Health, „Age of Money“, Runway, Vorschläge zum Auffüllen der Kategorien).

Goals



Goals (Flugzeug)

Oben rechts. Öffnet Sparziele: Name und Zielbetrag anlegen und verfolgen. Animation beim Tippen (Abflug).



Main Vault
420 €

Main Vault (Mitte)

Noch nicht zugewiesenes Geld („Ready to Assign“). Tippen öffnet **Move**, um Beträge in Kategorien zu verteilen.

4. Kategorie-Gebäude

Jedes Gebäude hat eine eigene Farbe. Tippen öffnet die Kategorie-Detailansicht.

Diner

Essen & Freizeit. Orange Akzentfarbe.

Home

Wohnen / Miete. Blauer Akzent.

Bills

Rechnungen & Abos. Violetter Akzent.

Travel

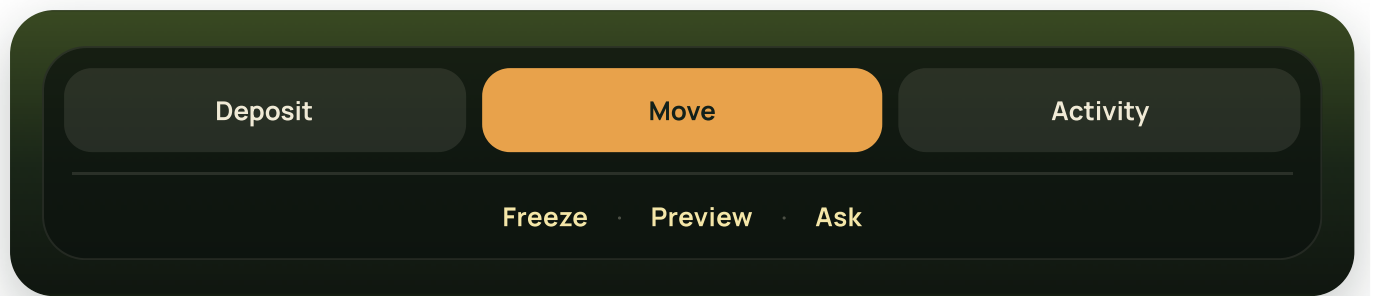
Mobilität / Reisen. Türkiser Akzent.

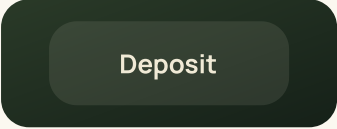
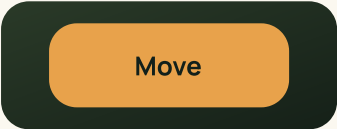
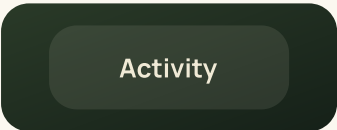
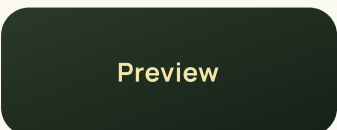
Groceries

Lebensmittel. Grüner Akzent. Anzeige: verfügbarer Restbetrag der Kategorie.

5. Untere Aktionsleiste (Dock)

Primäraktionen in der dunklen Leiste; darunter die drei Text-Links in Gold.



	Deposit Einkommen einbuchen. Der Betrag landet im Main Vault und steht zur Verteilung bereit.
	Move Hauptaktion (Ember-Orange). Geld vom Vault in eine Kategorie verschieben oder zwischen Kategorien umbuchen. Optional mit Notiz.
	Activity Chronik der letzten Einzahlungen und Verschiebungen – wer wohin wie viel bewegt hat.
	Freeze Betrag im Vault für einen Zeitraum sperren (Tag / Woche / Monat / individuell). Gesperrtes Geld zählt nicht als zuweisbar, bis die Frist abläuft.
	Preview Vorschau auf den nächsten Monat: Restbeträge in Gebäuden und typische wiederkehrende Zahlungen aus der Historie.
	Ask „Kann ich mir ... leisten?“ Auswertung aus Kontoständen, früheren Käufen und einer Preisschätzung (Katalog; optional Backend/KI).

6. Geldfluss (kurz)

1. **Deposit** → Geld in den Main Vault.
2. **Move** → Zuweisung vom Vault in Kategorie-Gebäude.
3. Ausgaben belasten den verfügbaren Betrag der Kategorie.
4. Monatsende: Rest oder Überziehung kann in den Folgemonat rollen.
5. **Freeze** hält Vault-Geld vorübergehend fest.

7. Backend

Die mobile App speichert Budget und Inselstand lokal auf dem Gerät (Zustand + AsyncStorage) und läuft vollständig offline. Das Backend ist optional und stellt Konto, Cloud-Backup sowie eine optionale Preisschätzung für Ask bereit.

Technik & Architektur

- **Laufzeit:** Node.js mit **Express** (HTTP-API, Port standardmäßig `4000`)
- **Datenbank:** PostgreSQL, Zugriff über **Prisma ORM**
- **Datenmodell:** Tabelle `User` (E-Mail, Passwort-Hash) und optional genau ein `Backup` -Datensatz pro Nutzer (JSON-Blob des kompletten App-Stands)
- **Konfiguration:** Umgebungsvariablen in `.env` (`DATABASE_URL` , `JWT_SECRET` , `PORT` , optional `OPENAI_API_KEY`)
- **CORS:** freigeschaltet, damit die Expo-App von Gerät/Simulator zugreifen kann
- **JSON-Limit:** bis ca. 5 MB, damit vollständige Backup-Snapshots übertragen werden können

Authentifizierung

- `POST /auth/register` – neues Konto (E-Mail + Passwort, min. 8 Zeichen). Passwort wird mit **bcrypt** gehasht gespeichert. Antwort: JWT + `userId`
- `POST /auth/login` – Anmeldung; bei Erfolg ebenfalls JWT (Gültigkeit 30 Tage) und `userId`
- Geschützte Routen erwarten Header `Authorization: Bearer <token>` . Prüfung über Middleware mit `jsonwebtoken` und `JWT_SECRET`

7. Backend (Fortsetzung)

Backup & Wiederherstellung

- `PUT /backup` – speichert den gesamten lokalen App-Stand (Kategorien, Transaktionen, Zuweisungen, Monat, Freeze-Sperren) als ein JSON-Objekt. Vorheriger Stand wird überschrieben (kein Merge).
- `GET /backup` – lädt den zuletzt gespeicherten Stand; ohne Backup: HTTP 404
- In der App erreichbar über **Account → Backup / Restore** (nur im eingeloggten Modus)
- Prinzip: die App bleibt die Quelle der Wahrheit; der Server speichert nur den Snapshot, ohne Budget-Geschäftslogik

Ask / Preisschätzung

- `POST /ai/price` mit Body `{ "item": "..." }`
- Zuerst optionaler Aufruf der OpenAI-API (wenn `OPENAI_API_KEY` gesetzt), Antwort als strukturiertes JSON (Preis, Spanne, Kurznotiz)
- Ohne API-Key oder bei Fehler: eingebaute **Produktkatalog**-Schätzung (typische DE/EU-Preise)
- Die mobile App kann Ask auch ohne Backend beantworten (lokaler Katalog + Historie); das Backend verbessert die Schätzung bei Bedarf

Weitere Endpunkte & Betrieb

- `GET /health` – einfacher Lebenscheck (`{ "ok": true }`)
- Start lokal: `npm install`, Prisma erzeugen/migrieren, dann `npm run dev`
- Die App spricht standardmäßig `http://localhost:4000` an; auf einem echten Smartphone muss die LAN-IP des Rechners in der API-URL eingetragen werden
- Ohne laufendes Backend bleiben Insel, Deposit, Move, Freeze, Preview, Ask (lokal) und Goals nutzbar

Rollenverteilung Client vs. Server

- **Client (Expo-App)**: YNAB-ähnliche Buchungslogik, Insel-UI, Freeze, Insights, Ask-Logik, Persistenz auf dem Gerät
- **Server**: Nutzerkonten, JWT, Speichern/Laden eines JSON-Backups, optionale Preis-API
- Kein Live-Sync bei jeder Buchung – bewusste, manuelle Sicherung über Backup/Restore